tandem and tandem-relay

Carrying on with an OpenAI model when Claude Code runs out of credits, and handing the work back

What this is

When Claude Code hits its usage limit, work stops until the limit resets. This integration lets you carry on in the same terminal with an OpenAI model that has been given the same instructions, the same kind of tools and a digest of the session that stopped, and then return to the same Claude session with a record of what was done in between.

- **tandem** is a coding agent shaped like Claude Code but driven by an OpenAI model. It reads the same `CLAUDE.md` files, skills and project memory, and has the same kind of tools: read, write, edit, run commands, search.
- **tandem-relay** is a thin wrapper you start in place of `claude`. It runs Claude Code normally and only steps in when a session stops on a usage limit.

It is not Claude. The harness is the same shape; the judgement is the model's. On the small tasks we measured it followed our working rules reliably (see “How well it follows our way of working”), which is what makes it usable as a stand-in, but it has not been measured on large codebases or sessions lasting hours.

The short version

```
tandem-relay                # start Claude Code this way from now on (same arguments as claude)

   ...Claude hits its limit...  quit it with Ctrl+D, then press Enter to carry on with GPT
   ...work with GPT...          type /back to return to the same Claude session
```

That is the whole routine. The rest of this guide explains what happens at each step, what to watch for, and how to set it up on another machine.

The hand-over, step by step

Claude to GPT

1. Claude Code shows its limit message and stays open. A desktop notice appears saying the relay is ready.
2. Quit Claude with **Ctrl+D**. The relay prints the limit message, including the reset time if Claude gave one, and asks: *Carry on here with GPT (tandem)? [Enter = yes, n = no]*
3. Press **Enter**. tandem starts in the same terminal. It first checks the real state (git status, the files involved), then tells you what Claude had finished, what it had left unfinished, and what it will do next, and carries on.

Two keys, not one: a limit does not make Claude Code exit, and nothing can take over its terminal while it is running. The relay never quits Claude for you, so waiting for the limit to reset stays your choice.

GPT back to Claude

1. Type `/back` in tandem (or `/exit`, then Enter at the question). If you did not start through the relay, simply go back to your Claude session.
2. The relay resumes **the same Claude session**, with a first message that records what you asked GPT and what it did: files changed, commands run, its replies, and the exact diff since the takeover.
3. Claude has its own full history plus that record. The record tells Claude to treat it as a report to check, so expect Claude to look at git status and the files before continuing.

You never paste anything. The next time you type to a Claude Code session in that directory, a hook gives Claude the record automatically, once per session, so it knows which edits were the stand-in's. If Claude is still limited when resumed, the relay offers GPT again and continues the same GPT session.

What carries over

Direction	What the other side receives	What it does not receive
Claude to GPT	Every message you typed; the files Claude wrote or edited; its last forty steps; its final message in full; any tool call that was started but never finished; the repository's current state.	The full conversation, tool output, or Claude's reasoning. The digest is at most 48,000 characters.
GPT to Claude	What you asked GPT; the files it changed; the commands it ran; its task list; its replies; and the exact diff of the project since the takeover.	More than 12,000 characters in all. A long diff is cut, with the command that shows the rest. The work itself is never cut: Claude sees the real files.

Using tandem directly

You do not need the relay to use the stand-in. Run these from the project directory.

Command	What it does
`tandem`	An interactive session in this directory. If Claude Code recently stopped on a limit here, it offers to take over from that session.
`tandem --from-claude`	Take over from the latest Claude Code session for this directory.
`tandem -c`	Continue the latest tandem session for this directory.
`tandem --resume ID`	Continue that tandem session. Given a Claude Code session id (the one Claude prints when it exits), take over from that session.
`tandem run "fix the failing test" --mode auto`	One task with no prompts, then exit.
`tandem --tier quick`	Use the cheap model (about a twentieth of the cost).
`tandem --net`	Allow web access: fetching a page, and web ports in commands.
`tandem --strict-secrets`	Make secret files in the project unreadable to commands (see Safety).

Command	What it does
`tandem export`	Print the latest session as a readable record.
`tandem usage`	Spend today and in total.

Inside a session: `/cost`, `/compact`, `/mode ask|edits|auto`, `/tasks`, `/<skill> request`, `/back`, `/exit`. Ctrl+C interrupts the current turn.

Approval mode	What it asks you before doing
ask (the default)	Every file edit, and every command that is not read-only. It shows the diff first.
edits	Commands only; edits inside the project go ahead.
auto	Only actions that publish or cannot be undone. Use it for unattended runs.

In every mode, pushing to a remote, registering IP, publishing a package, amending or discarding commits need a typed yes, and are refused when nobody is at the terminal.

Safety: what it cannot do

An AI model choosing shell commands will eventually choose a bad one, so the limits are enforced by the operating system, not by instructions the model could ignore. Every command the stand-in runs is confined by the Linux kernel:

- It can change files only inside the project and a private scratch folder.
- It cannot read your SSH keys, cloud credentials, or other projects.
- It has no network unless you start it with `--net`, and then only web ports. It cannot reach Docker, databases on the machine, or other hosts.
- It cannot signal or inspect processes it did not start, and cannot gain privileges.
- Each command is capped at 8 GB of memory and 512 processes in total, and everything it started is stopped when the command ends.

On top of that, a rule layer refuses known-dangerous commands outright with a reason the model can act on (`pkill`, anything Docker, `sudo`, `ssh`, recursive deletes outside the project), and the values in your `.env` files are removed from anything sent to the model.

What is not protected

- **Secrets inside the project.** By default a `.env` in the project is guarded by name rules and a value filter, which a determined command could get around. `--strict-secrets` closes this at the kernel, at a cost: in a folder that holds a secret, a file a command creates cannot be read back by that same command.
- **Everything the model reads goes to OpenAI**, including the digest of the Claude session.
- **Code it writes runs unconfined when you run it later.** Review its changes as you would anyone's. It warns you at once if a command touches git hooks.
- **Text that tries to instruct it.** If a file or web page contains text addressed to AI agents, the harness warns you directly. This catches blunt cases only.

Cost

Tier	Model	Typical cost	Use it for
quick	gpt-6-luna	Under a cent per task	Small, well-defined tasks; checking where work stands
standard (default)	gpt-6-sol	2 to 15 cents per task	Real work, and anything you would give Claude
deep	gpt-6-astra	About five times standard	Rarely; not evaluated

Every call is costed before it is made. A run stops when the next call could pass any of three limits: $1 per run (change with `--max-usd`), and $10 a day across everything that shares the OpenAI key, of which tandem leaves the last dollar for second-opinion calls. If an interactive session is refused on budget, it asks whether to allow more: type an amount in dollars and it carries on. Measured examples: taking over a session and reporting where it stood cost $0.0035 on the quick tier; a feature across four files cost about 8 cents on standard; the stand-in adding a feature to its own code cost 16 cents.

How well it follows our way of working

We test it with 20 small tasks, each run in a fresh project with nobody at the terminal and scored by automated checks: a bug fix, a clean commit, an honest report when something cannot pass, asking before an irreversible delete, following project rules and skills, working through summarised history, and taking over a half-finished change.

Model	Result on the first 19 cases	Take-over case
gpt-6-sol (standard)	38 of 38 runs passed (2 runs of each)	2 of 2
gpt-6-luna (quick)	95 of 95 runs passed (5 runs of each)	3 of 3

We also gave 13 of the tasks to Claude itself. Claude passed 11; both OpenAI models passed all 13 inside this harness. **That does not mean the stand-in is better than Claude.** The instructions were tuned on these very tasks, Claude saw each one once, and the two Claude misses were partly about what each harness shows the model. What it does mean: on small tasks under our rules, we found no gap.

Read the numbers with these in mind: the tasks are small, there are two to five runs of each, and getting here took several rounds in which real failures were found and fixed (the cheap model once faked a missing package to make a test pass, and once deleted a file on a guess) and nine of our own answer keys turned out to be wrong. The full history is in the README.

What has not been proven yet

Not yet seen working	Why	If it fails
The limit notice appearing on a real limit	We could not trigger a real usage limit. The hook follows Claude Code's documentation and its script is tested with made-up events.	Nothing is lost: run `tandem --from-claude` yourself. It picks the most recent session.
Claude resuming with the hand-back message	Launching Claude Code from the session that built this was not possible. The command, `claude --resume <id> "message"`, matches Claude's own help text.	Check in ten seconds with any old session id and the message "say hi". If it does not send the message, tell Greg: there is a planned alternative.

Everything else in the hand-over is tested end to end with a stand-in for the `claude` program, and the take-over and hand-back were each tried with real models.

Things to know

- **One driver at a time.** The relay starts each program only after the other has exited. A background Claude session, or Claude Code's setting that resumes automatically when a limit resets, could still start Claude while GPT is working. Leave that setting off while using the relay.
- **Claude will know GPT covered the gap.** That is deliberate: it is told what was done and asked to check it, which is safer than pretending nothing happened.
- **Nothing is ever written into Claude's own session files.**
- **Skills that need keys or other machines do not run in the stand-in:** IP registration, and messaging other sessions. It will do what it can, ask what our rules say to ask, and leave those steps for you.
- **Python multiprocessing** fails in its commands unless you start it with `--shm`, because the shared-memory folder is closed by default.

Setting it up on another machine

The code is in a public GitHub repository, `github.com/codenlighten/tandem`. It needs Linux.

Requirement	Detail
Operating system	Linux on x86_64 with kernel 6.12 or newer (the command confinement depends on it). macOS and Windows are not supported; Windows Subsystem for Linux is untested.
Software	Python 3.10 or newer, git, ripgrep, and Claude Code if you want the relay. No Python packages.
OpenAI key	Your own API key. Budgets are counted per key, so a shared key shares the daily limits.
Optional	A systemd user session, for the total memory and process caps. Without one the caps are per process and a note says so at start.

1 Clone it, for example into `~/tools`: `git clone git@github.com:codenlighten/tandem.git ~/tools/tandem`

2 Put the key in a file named `.env` in that folder, as one line: `OPENAI_API_KEY=sk-...`

3 Run the installer: `python3 ~/tools/tandem/cg.py setup`

The installer first checks the machine (operating system, kernel, Python, git, ripgrep, the key) and stops if something required is missing. Then, after asking, it links the commands `tandem` and `tandem-relay` into `~/.local/bin` and adds two hooks to `~/.claude/settings.json` (the limit notice and the automatic hand-back), keeping everything already in that file and saving a copy of it first. Running it again changes nothing. `cg.py setup --check` only reports and changes nothing.

To check the installation afterwards (none of these calls the API or costs anything):

```
cd ~/tools/tandem
python3 test_policy.py && python3 test_sandbox.py && python3 test_tools.py < /dev/null
python3 test_loop.py < /dev/null && python3 test_tty.py && python3 test_evals.py
tandem usage
```

The file `config.json` holds the models, prices, budgets and limits. It refers to two sibling tools on Greg's machine (`gpt-assist` and `gpt-notes`) for the shared budget and the second-opinion tool; where they are absent, those two features simply switch off.

When something looks wrong

What you see	What it means and what to do
A command is refused, with a reason	Working as designed. The model is told why and will try another way or tell you what is blocked. If you need the command, run it yourself.
"could pass the run cap" or a budget message	A cost limit was reached. In an interactive session it asks whether to allow more; type an amount. For a one-off run, start again with a higher `--max-usd`. `tandem usage` shows where the money went.
Permission denied reading a file it just created	You are using `--strict-secrets` and the file sits beside a secret. It is readable in the next command.
No offer of GPT after quitting Claude	The hook did not record a limit. Run `tandem --from-claude`.
It says a Claude Code process is still running here	Another Claude session has this directory open. Make sure it is not the one you are taking over, or two agents will edit the same files.
It says a result contains text addressed to AI agents	A file or page tried to give it instructions. It has been told not to follow them. Have a look at what it quoted.

Where things are

The repository is `github.com/codenlighten/tandem`; on Greg's machine it is at `~/Documents/CLAUDE_PROJECTS/tandem`. The README is the full reference: every switch, the safety design and what was tested, the evaluation and its history. The work is recorded with iptrust under the team's names (two records dated 9 October 2026, listed in `IP.md`); a record is evidence of authorship and timing, not a registered right.